An Overview of zkTLS & DECO

Alyssa Brittany Chen

May 9, 2025

Contents

1 Introduction **2**
- 1.1 TLS 1.2 Handshake . . . 2
- 1.2 Record Protocol . . . 4
- 1.3 Zero-Knowledge Proofs . . . 4
- 1.4 Oracles . . . 5
- 1.5 The Problem . . . 5

2 zkTLS **5**
- 2.1 TEE-TLS . . . 5
- 2.2 MPC-TLS . . . 6
- 2.3 Proxy-TLS . . . 7

3 DECO **9**
- 3.1 Three-party Handshake . . . 10
  - 3.1.1 Key Derivation . . . 11
- 3.2 Query Execution . . . 11
- 3.3 Commit and Verify . . . 12
- 3.4 Proof Generation . . . 12
- 3.5 Applications & Challenges . . . 12

Abstract

zkTLS is a cryptographic primitive that enables verifiable Transport Layer Security (TLS) communication without revealing sensitive session data. By combining traditional TLS with zero-knowledge proofs, zkTLS enhances privacy while preserving the authenticity of the underlying data to third parties.

We will explore three main approaches to implementing zkTLS:

1. MPC-based protocols, which distribute trust across multiple parties
2. Proxy-based solutions, which mediate TLS sessions through a zk-proof-generating proxy
3. TEE-based designs, which leverage trusted hardware for secure execution and attestation

Lastly, the paper will include a discussion and walk-through of the DECO whitepaper: Liberating Web Data Using Decentralized Oracles for TLS.

1 Introduction

1.1 TLS 1.2 Handshake

Diffie Hellman is a cryptographic key exchange, one of the first symmetric key exchange protocols implemented and widely used. One of its applications is in securing the internet using https, which is based on TLS. TLS defines the fact that two parties should perform a DH key exchange before encrypting data. The Diffie-Hellman exchange is responsible for setting up the secret key for encryption using the TLS Handshake before encrypted communication occurs.

The TLS handshake protocol involves two parties: the client and server. In this case, the client would typically be a browser, and the server would be some domain or website being visited.

Whenever a user visits any website, the browser initiates a **ClientHello** messsage request to the domain, along with a random nonce, a list of supported cipher suites with encryption schemes the client supports. This typically looks like AES or Chacha20, two of the most widely used encryption schemes that are part of the TLS 1.3 standard.

Upon recieving the **ClientHello** message, the server becomes aware the client wants to engage in a TLS handshake. It will then check the nonce and select a compatible cipher suite and key exchange method, sending a **ServerHello** along with the chosen encryption scheme. Subsequently, the server sends a digital server certificate containing a public key and a certificate chain signed by a trusted Certificate Authority (CA). It is important to note that the certificate also issues a private key that is to be securely stored.

Now that the client has the certificate, it checks for two things in order to verify it is talking to the correct server:

1. Does the certificate match the domain it is trying to connect to?
2. Is the CA authority that issued the certificate a trusted authority?

Up until this point, the client and server are still trying to decipher if they should even engage in a Diffie-Hellman key exchange. If at this point, the certificate issued is not a trusted certificate, then the client closes the connection. If the connection remains open, that indicates the certificate is trusted, and the server begins to engage in a server key exchange.

The server key exchange picks a random generator g, and sends that across to the client. Randomly, the server selects a private key b (different than the one above) and generates an ephemeral public key $g^b \bmod p$, sending the public key, the large prime p, and a signature to the client.

An example of a signature that uses RSA: $signature = Sign_{priv}(Hash(g, g^b, p, context)$

The signature proves the key comes from the entity that owns the certificate. Without the private key from above, a malicious entity cannot produce a valid signature, which is integral to proving authenticity.

The client will then pick its own private key, a and a **ClientKeyExchange** message, sending sends its own ephemeral public key $g^a \bmod p$ to the server, where g, p are the same from the server key exchange. The client and the server then establishes a shared secret, $g^{ab} \bmod p$ and uses this to derive symmetric encryption keys for the session.

Finally, the client and server send a **Finished** message. Once this exchange is successful, the TLS session is established and secure communication can begin. The client can start encrypting data or private information– for example, submitting credit card details during an online purchase.

Note: this example illustrates the TLS 1.2 protocol using a Diffie-Hellman based approach. In TLS 1.3, many handshake steps are eliminated, and uses only ephemeral key exchanges (e.g. X25519, ECDHE).

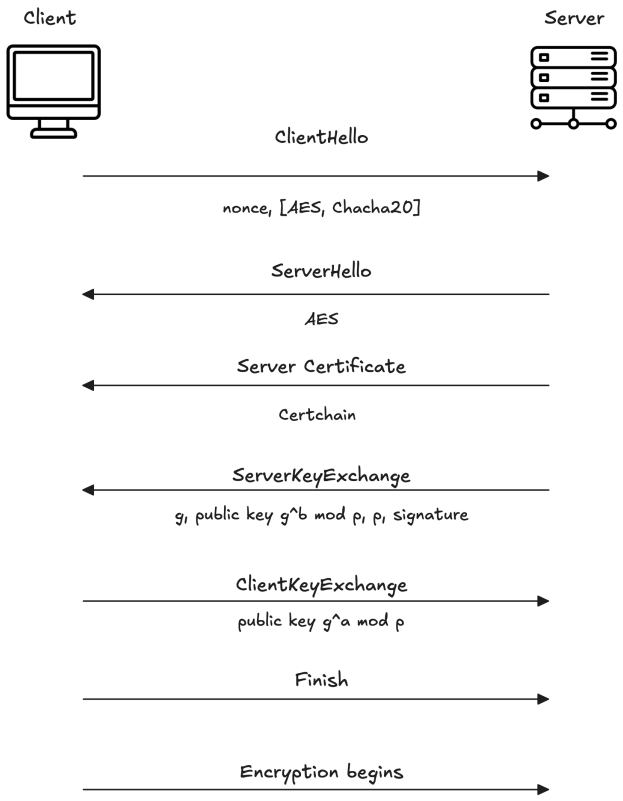


Figure 1: TLS Handshake (1.2) Diagram

1.2 Record Protocol

While the handshake protocol exchanges messages across client and server, it ultimately produces a secret key for symmetric encryption. The record protocol uses this secret key to encrypt outgoing messages and decrypt incoming messages with the same secret key. This verifies that the messages have not been tampered with and allows for secure communication of the encrypted messages.

1.3 Zero-Knowledge Proofs

Zero-knowledge proofs were first introduced in Goldwasser-Micali-Rackoff's paper titled "The Knowledge Complexity of Interactive Proof Systems" (1985). A zero-knowledge proof is a way to demonstrate something is true without revealing any of the underlying details.

They should satisfy the following three properties:

1. Completeness: If the prover was honest, the verifier will be convinced,
2. Soundness: If the prover was not honest, they would fail to convince the verifier (with prob. $\geq 1 - \epsilon$)
3. Zero Knowledge: The verifier cannot obtain any extra information

1.4 Oracles

Oracles are entities that connect blockchain networks or smart contracts (self-executing contracts via code) to access existing data in external systems. They act as intermediaries to deliver data to third parties and operates while keeping sensitive data hidden. There are a few different types of oracles:

1. Input Oracles, which retrieves data from off-chain and delivers it onto the blockchain network.
2. Output Oracles, which allow smart contracts to send requests to systems off-chain to execute an action.
3. Cross-chain Oracles, that read and write information across various blockchains.

1.5 The Problem

Currently, because TLS is designed to only act between a client and server, a lot of data is locked up in the internet. If the data were to be shared with a third party, the traditional TLS model has no way of proving the data came from a particular website. Therefore, users' private data is often non-accessible outside this point of origin.

For example, let's say Alice wanted to prove to Bob she is older than 18. Typical age-verification services require ID verification or to upload sensitive information. Other websites (such as the DMV) store verified birth dates. Alice could in principle just send a screenshot of her birth date from the DMV site, but with current technology, this information can be forged trivially. However, even if there existed a way to prove the authenticity of the screenshot, it would reveal information other than just her birth date– therefore this method has limitations and is not secure.

2 zkTLS

Zero-knowledge transport layer security (zkTLS) is a protocol that allows for information to flow securely between multiple parties. While TLS keeps data secure during transmission between server and client, it fails to prove the authenticity and integrity of the data packages. By utilizing zk proofs, we are able to prove the authenticity of private information without revealing sensitive information.

Many modern zkTLS frameworks are built to be added to existing TLS protocols, which makes it easy for developers to implement.

2.1 TEE-TLS

Instead of running the browser on a desktop environment, TEE-TLS runs it inside a **Trusted Execution Environment**, such as Intel SGX or AWS Nitro Enclaves. When the TLS handshake is executed within a TEE, two key security properties are guaranteed:

1. Privacy: any computation being done in TEE (including the data being generated) is kept private. Even the owner of the physical TEE does not have access to the data being generated.
2. Integrity: The code running inside the TEE is guaranteed to be exactly the code that was provisioned and attested.

Drawback: You would have to trust the hardware vendor (e.g., Intel or AWS) that the TEE is working as expected. There have been multiple times this assumption have been broken (e.g, side-channel attacks) and have compromised TEE guarantees.

2.2 MPC-TLS

Multiparty computation (MPC) is a cryptographic method which allows multiple users to jointly make computations over their private data. MPC-TLS introduces a third party, a notary, in addition to the server and client. The client and server engage in **ClientHello, ServerHello, ServerCert** and send over the generator, public key, and signature as normal.

Then, the notary and client will engage in multiparty computation (MPC) and together generate a private key a. However, an important aspect to note is that neither the notary nor the client has full access to a– this is a property of MPC. Both the notary and the client holds their own shares of the private key, and together can reconstruct a.

Later, when sending encrypted requests, the client and notary again collaborate using their shared key parts to produce valid, authenticated messages. The notary also signs the response and sends it to a third party for verification.

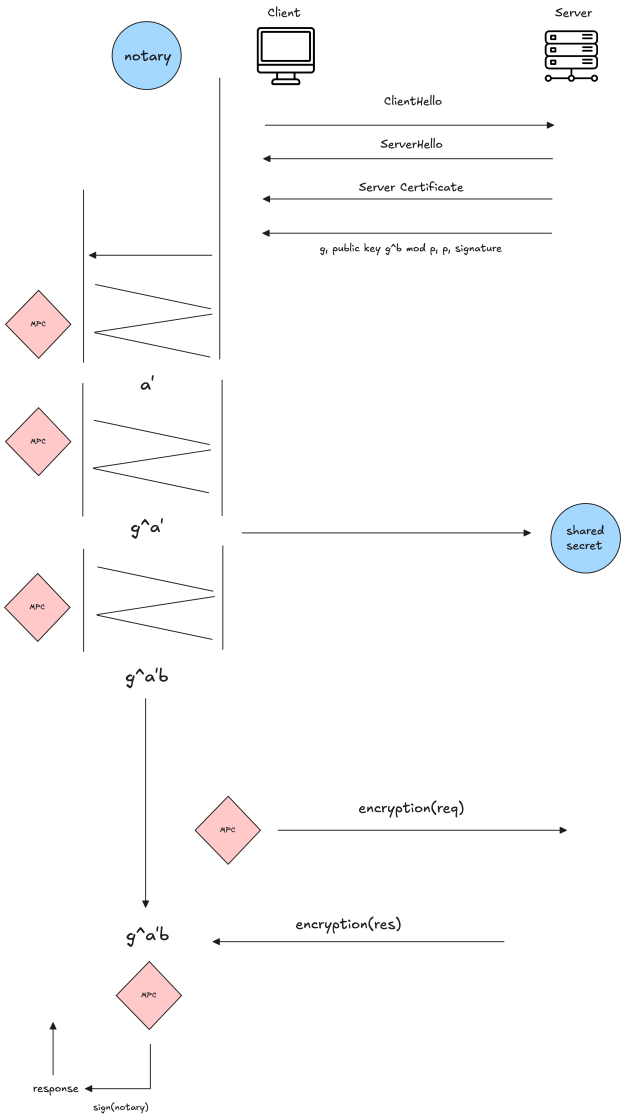


Figure 2: MPC-TLS Diagram

Drawbacks: Multiple rounds of MPC are involved in this model, which can be computationally intensive and introduces latency, downgrading user experience. Despite this, MPC-TLS is generally the most secure out of the three approaches due to its strong privacy guarantees within the distributed trust assumptions.

2.3 Proxy-TLS

Similar to MPC-TLS, Proxy based TLS introduces a third party– a proxy. The client and server engage in TLS like normal, and once the handshake is complete, the client sends over an encrypted request and receives an encryption response. All of the data during this session is routed through the proxy.

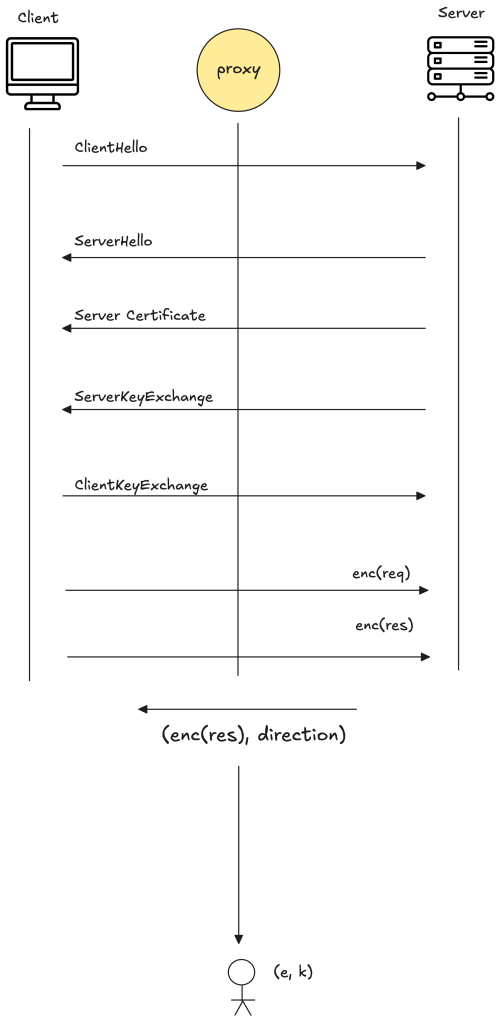


Figure 3: Proxy-TLS Diagram

Since the proxy does not participate in the key exchange, it does not know the shared secret. As a result, it cannot read the encrypted data. However, it can attest that it observed the encrypted data flowing from the server to the client and vice versa.

Proxy-TLS is a much simpler and more efficient construct– it is also easier to implement than the MPC approach. It is potentially vulnerable to man-in-the-middle (MITM) attacks, where an attacker could feed malicious encrypted messages to the proxy. Because the proxy cannot decrypt the content, it cannot verify the message's authenticity or source—only the direction.

However, executing a successful MITM attack against Proxy-TLS typically requires being on the same physical or network layer as the client or server, which is a high barrier in most threat models. Thus, the proxy based approach remains practical for most use cases where performance is valued.

3 DECO

DECO (short for decentralized oracle) is a research paper initially published in 2020 and presented at CCS '20 (Computer and Communications Security). In order to address the provenance issue in TLS– that data comes directly from a particular website, DECO allows for this proof without any trusted hardware assumptions or server end modifications. Additionally, the user is also able to prove statements in a zero-knowledge manner.

DECO utilizes zero-knowledge proofs, incorporating this with their oracles to enable users to prove web session details without revealing the session data itself to the oracle.

Current oracle schemes require website-specific support. DECO offers a source-agnostic implementation, and is able to support any website running TLS. Additionally, DECO does not require server-side cooperation.

The prover commits data D and proves to the verifier that D came from a TLS server S, and can also optionally prove a statement π_D about the verifier. In the age example, π_D could be in the form $D = y/m/d$, which is Alice's birth date and compares it with the current date to prove Alice is at least 18 years of age.

As we have highlighted previously with the TEE-TLS approach, trusted hardware has its limitations because it has been shown to be vulnerable to attacks. DECO achieves two main properties as highlighted below:

1. Authenticity: The verifier is convinced only if the asserted statement about D is true and D is indeed obtained from website S.
2. Privacy: Only the verifier learned the statement π_D holds for some D obtained from S.

Additionally, because TLS generates symmetric encryption keys, the client can easily forge arbitrary TLS session data. DECO introduces a three-party handshake protocol (prover, verifier, web server) with an unforgeable commitment on a piece of data D in the TLS session. DECO is compatible with both TLS 1.2 and 1.3 versions, and is the first modern oracle scheme that does not rely on trusted hardware assumptions and server-side modifications.

3.1 Three-party Handshake

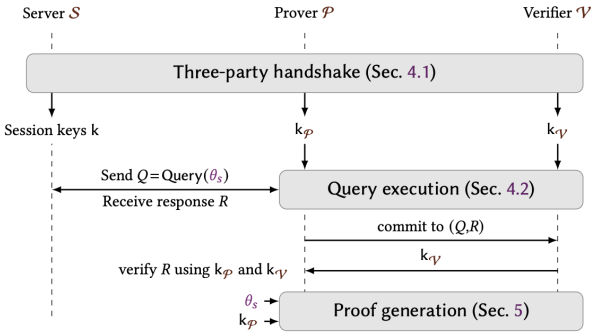


Figure 2: An overview of the workflow in DECO.

Figure 4: Three Party Handshake Diagram (Source: DECO)

Three-party handshake follows two steps as with the standard TLS handshake. DECO highlights Elliptic-curve Diffie-Hellman (ECDHE) as the TLS-compatible key exchange protocol. We can let $EC(\mathbb{F}_p)$ be the group used in ECDHE and its generator G. The steps are then as follows:

1. Prover P initiates handshake, sending ClientHello and r_c, a random nonce to server S.
2. Prover receives server certificate, server nonce r_s, and $Y_S = s_S \cdot G$, the signed ephemeral DH public key from S.
3. P checks the signature and forwards to V.
4. V samples secret s_V, sending her share of DH public key $Y_V = s_V \cdot G$ to P. P then samples s_P and sends combined public key $Y_P = s_P \cdot G + Y_V$ to S.
5. S then computes their share of the DH secret as $Z = s_S \cdot Y_p \cdot P$, while P computes their share of Z as $Z_p = s_P \cdot Y_S$ and $Z_V = s_V \cdot Y_s$. Here, we note that the group operations of $EC(\mathbb{F}_p)$.

At a high level, the motivation for introducing a multi-party handshake protocol is to divide the user into two roles– a prover and a verifier. The prover and verifier jointly establish a TLS session with the server in a way that generates a cryptographic proof about the servers response.

Feature	Standard TLS (2-party)	DECO TLS (3-party)
Participants	Client, Server	Prover, Verifier, Server
Session Key Ownership	Known only to Client & Server	Split between Prover & Verifier (via MPC)
Verifiability by Third Party	✗ Not possible	✓ Zero-knowledge proof generation possible
Server Modification Required	✗ No	✗ No
Data Privacy	✓ Yes (encrypted)	✓ Yes (selective disclosure)
Proof of Session Contents	✗ No	✓ Yes (to verifier or smart contract)
Smart Contract Compatibility	✗ Not designed for on-chain use	✓ Compatible with on-chain oracles
Attack Surface	Minimal (2 actors)	More complex (requires secure MPC or TEE)

Table 1: Comparison of Two-Party and Three-Party TLS Handshakes

3.1.1 Key Derivation

The additive shares of Z are now in the form of elliptic curve points, and now we proceed to derive keys. Here, the paper highlights the technical challenges of using boolean circuits to evaluate arithmetic in large fields. Elliptic curve point addition involves about 4 subtractions, one modular inversion and 2 modular multiplications, which is estimated to result in over 900,000 AND gates for calculations (not including the inversions, where we would have to run the Extended Euclidean algorithm inside a circuit).

ECtF (Elliptic Curve to Field) is a conversion protocol that allows parties to jointly convert a secret-shared elliptic curve point into secret-shared bits of its x-coordinate in a finite field $\mathbb{F}_p$, without revealing the point itself. DECO proposes using the ECtF protocol to convert additive shares of an elliptic curve point in $EC(\mathbb{F}_p)$ to additive shares of its x-coordinate in $\mathbb{F}_p$, which only involves using about $3|p|$ AND gates, where p is 256-bit. This algorithm will be outside of the scope of this paper, but its implementation can be read further in the original DECO paper.

3.2 Query Execution

The Prover P will compute a query following $Q = Query(\theta_s)$. P then invokes 2PC-HMAC (two party Hash-Based Message Authentication Codes) with the verifier, computing τ, which is the HMAC tag of the query Q. DECO mostly focuses on one-round query sessions, but also discusses multi-round sessions in their appendix.

$\text{HMAC}(k, m) = H((k \oplus \text{opad} \,||\, \text{H}((k \oplus \text{ipad} \,||m)$ is the HMAC of message m, where H denotes SHA-256. DECO optimizes the computation involved in 2PC by designating the inner hash of $\text{HMAC}(k, m)$, which is $\text{H}((k \oplus \text{ipad} \,||m)$, to be local to P, and by exploiting

the Merkel-Damgard structure in SHA-256. After, P sends $\text{sid}, \hat{Q} = \text{Enc}(\text{k}^{\text{Enc}}, Q||\tau))$ to S, where sid denotes the session id, and $\hat{Q}$ denotes the encrypted query which includes the original query Q, tag τ and key k.

Remark: The final encrypted $\hat{Q}$ sent to the server contains not just the original query Q, but also its HMAC tag τ which is encrypted under k^{Enc}. This is designed such that the server cannot observe anything in plaintext, but the encrypted result can still be used in proof generation later on.

3.3 Commit and Verify

After P receives the response $\text{sid}, \hat{R}$, P then sends a commitment to the session data to V. This commitment is denoted $\text{sid}, \hat{Q}, \hat{R}, k_P^{\text{MAC}}$, where MAC denotes a message authentication code computed as $\text{MAC}(k, m)$. V then sends back $\text{sid}, k_V^{\text{MAC}}$ to P, which allows P to compute $k^{\text{MAC}} = k_V^{\text{MAC}} + k_P^{\text{MAC}}$. This allows P to decrypt $R||\tau = \text{Dec}(k^{\text{Enc}}, \hat{R})$ and verifies τ against the key, k^{MAC}.

This step ensures the prover cannot retroactively change their query. When P sends in their commitment, P binds themselves to the session data before seeing the verifier's final MAC key share.

3.4 Proof Generation

The scope of proof generation is outside the contents of this paper– refer to section of 5 of the DECO whitepaper.

3.5 Applications & Challenges

1. Confidential financial instruments, such as binary options.
2. Age proofs.
3. Reporting price discrimination of advertised goods while hiding sensitive information (i.e. name, address).

While DECO is able to provide delivery of private data for third parties– and optionally for public use, this capability raises legal and regulatory challenges since users can retrieve data from websites without their awareness. These compliance issues raise an interesting perspective to consider for users and websites affected about this technology.

References

[A] Reclaim Protocol. TLS Handshake

[B] Fan Zhang et al. DECO: Liberating Web Data Using Decentralized Oracles for TLS

[C] Reclaim Protocol. Claiming and Managing Self-Sovereign Credentials

[D] Zhongtang Luo et al. Proxying is Enough: Security of Proxying in TLS Oracles and AEAD Context Unforgeability

[E] TLS Notary Documentation

[F] Chainlink. What is a Blockchain Oracle?